



## سیستم‌های عامل

نیم‌سال دوم - سال تحصیلی ۱۴۰۴ - ۱۴۰۵

استاد درس

دکتر طاهری جوان

پروژه پایانی

## پروژه پایانی



دانشگاه صنعتی امیرکبیر • دانشکده مهندسی کامپیوتر



بهار ۱۴۰۵

### قوانین و اطلاعات کلی پروژه

- دانشجویان باید مخزن `xv6-riscv` را `fork` یا `clone` کرده و در طول انجام پروژه، تاریخچه کامیت‌های گیت‌هاب خود را حفظ کنند.
- فایل ارسالی در سامانه باید حتماً با فرمت `Proj_StuId.zip` باشد و تنها شامل فایل‌های تغییر یافته پروژه `xv6` به همراه فایل گزارش کار با فرمت **PDF** باشد.
- گزارش کار باید شامل توضیح تغییرات اعمال شده، معماری و طراحی راه حل، فایل‌های تغییر یافته، نحوه پیاده‌سازی بخش‌های مختلف پروژه، نتایج اجرای برنامه‌های تست و تحلیل خروجی آن‌ها باشد.
- انجام این پروژه به صورت انفرادی است.
- مهلت ارسال پروژه: ۳۱ خرداد.
- پروژه دارای جلسه دفاع حضوری/آنلاین هستند. تمام اعضای گروه باید تسلط کافی داشته و قادر به توضیح و تفسیر کدهای نوشته شده باشند.
- بخش پایانی پروژه (Lottery Scheduling) به صورت امتیازی در نظر گرفته شده است.
- دستیاران آموزشی : جهت رفع اشکال و پرسش سوالات فنی، می‌توانید از طریق تلگرام یا بله با آیدی‌های @mahanzavari و @javad\_akb در ارتباط باشید.

## الزام نگارش برنامه‌های تست

- دانشجویان موظف‌اند برای اثبات صحت عملکرد کدهای خود در هر یک از بخش‌های پروژه (بخش اول، دوم و سوم)، برنامه‌های کاربردی اجرایی مجزایی در فضای کاربر (User-space) بنویسند (به عنوان مثال فایل‌هایی نظیر `test_prio.c` یا `test_lottery.c`).
- این برنامه‌ها باید سناریوهای مختلفی را شبیه‌سازی کنند (مانند ایجاد همزمان چندین فرآیند فرزند با اولویت‌ها یا بلیت‌های مختلف) و با چاپ خروجی‌های خوانا در ترمینال، کارکرد درست زمان‌بند یا فراخوان‌های سیستمی را نشان دهند.
- در جلسه دفاع، اجرای همین برنامه‌های تست در محیط xv6 و تحلیل خروجی آن‌ها، معیار اصلی ارزیابی شما خواهد بود.

## قوانین مربوط به اصالت کد و کپی‌برداری

- استفاده از ابزارهای هوش مصنوعی (نظیر Gemini یا Claude) تنها برای درک مفاهیم و دیباگ مجاز است. کپی‌پیست کردن مستقیم کدها به شدت ممنوع است.
- توانایی شما در پاسخگویی به سوالات شفاهی در جلسه دفاع، اثبات‌کننده اصالت کار شماست.

## فهرست مطالب

|    |     |                                                                                                   |
|----|-----|---------------------------------------------------------------------------------------------------|
| ۵  | ۱   | مقدمه و آشنایی با <b>xv6 RISC-V</b>                                                               |
| ۵  | ۱-۱ | نیازمندی‌های راه‌اندازی . . . . .                                                                 |
| ۶  | ۲   | نحوه اجرای <b>xv6</b> در محیط‌های مختلف                                                           |
| ۱۰ | ۳   | بخش اول: افزودن <b>System Call</b> جدید                                                           |
| ۱۱ | ۴   | بخش دوم: پیاده‌سازی <b>Priority Scheduling</b>                                                    |
| ۱۳ | ۵   | بخش سوم: پیاده‌سازی <b>Lottery Scheduling (Bonus)</b>                                             |
| ۱۵ | ۶   | معیارهای پذیرش و بارم‌بندی ( <b>Acceptance Criteria</b> )                                         |
| ۱۵ | ۱-۶ | بخش اول: فراخوان سیستمی <code>getpinfo</code> و برنامه کاربردی <code>ps</code> (۶ نمره) . . . . . |
| ۱۵ | ۲-۶ | بخش دوم: زمان‌بندی مبتنی بر اولویت و برنامه کاربردی <code>chpri</code> (۱۰ نمره) . . . . .        |
| ۱۶ | ۳-۶ | بخش سوم (امتیازی): زمان‌بندی قرعه‌کشی (Lottery) (۴+ نمره بونوس) . . . . .                         |
| ۱۶ | ۴-۶ | معیارهای عمومی، گیت و جلسه ارائه (۴ نمره) . . . . .                                               |
| ۱۷ | ۷   | منابع پیشنهادی                                                                                    |

# ۱ مقدمه و آشنایی با xv6 RISC-V

سیستم‌عامل xv6 یک پیاده‌سازی آموزشی و مدرن از نسخه ششم یونیکس (Unix V6) است که توسط دانشگاه MIT توسعه یافته و در این نسخه برای معماری RISC-V بازنویسی شده است. این سیستم‌عامل با وجود سادگی، بسیاری از مفاهیم کلیدی سیستم‌عامل‌های واقعی نظیر مدیریت حافظه، زمان‌بندی پردازنده‌ها، فراخوان‌های سیستمی (System Calls) و سیستم فایل را دارا می‌باشد. در این پروژه، شما هسته (Kernel) سیستم‌عامل xv6 را تغییر داده و امکانات جدیدی به آن اضافه خواهید کرد. هدف این است که با اضافه کردن فراخوان‌های سیستمی جدید و تغییر الگوریتم زمان‌بندی پیش‌فرض سیستم، با نحوه تعامل فرآیندها با هسته و مدیریت منابع پردازنده آشنا شوید.

## ۱-۱ نیازمندی‌های راه‌اندازی

برای کامپایل و اجرای سیستم‌عامل xv6-riscv، سیستم شما باید مجهز به دو ابزار پایه‌ای باشد:

۱. کامپایلر متقاطع معماری **RISC-V** (مانند ابزار `riscv64-unknown-elf-gcc` یا معادل‌های کاربردی آن).

۲. شبیه‌ساز **QEMU** با قابلیت اجرای شبیه‌سازی سیستم‌های RISC-V (ابزار `qemu-system-riscv64`).

کد منبع اصلی و خام این پروژه از طریق **مخزن گیت‌هاب رسمی xv6-riscv** در دسترس شما قرار دارد.

## ۲ نحوه اجرای xv6 در محیط‌های مختلف

بسته به سیستم‌عاملی که برای توسعه انتخاب می‌کنید، دستورالعمل‌های راه‌اندازی زنجیره ابزار (Toolchain) متفاوت خواهد بود. در ادامه نحوه نصب در توزیع‌های مختلف لینوکس، مک و ویندوز آورده شده است.

### ۱ محیط Ubuntu / Debian

ابتدا مخازن سیستم را به‌روزرسانی کرده و بسته‌های مورد نیاز را نصب کنید:

```
sudo apt update sudo apt install git build-essential gdb-multiarch  
qemu-system-misc gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu
```

پس از نصب ابزارها، مخزن xv6 را شبیه‌سازی کرده و وارد پوشه پروژه شوید:

```
git clone https://github.com/mit-pdos/xv6-riscv.git cd xv6-riscv
```

پروژه را کامپایل کرده و سیستم‌عامل را تحت شبیه‌ساز بوت کنید:

```
make qemu
```

بهتر از قبل از هر بار بوت کردن سیستم عامل از دستور زیر برای پاک کردن فایل‌های ساخته شده قبلی استفاده کنید.

```
make clean
```

عیب‌یابی محیط اوبونتو: در صورتی که حین اجرای فرآیند با خطای عدم شناسایی دستور `qemu-system-riscv64: command not found` مواجه شدید، ابتدا وضعیت و نسخه شبیه‌ساز نصب‌شده روی سیستم را با دستور زیر بررسی کنید:

```
qemu-system-riscv64 --version
```

در نسخه‌های جدید سیستم‌عامل اوبونتو، این شبیه‌ساز به صورت خودکار همراه با بسته `qemu-system-misc` نصب می‌شود. برای هماهنگی کامل، ترجیحاً از نسخه‌های پایدار

شبه‌ساز (نسخه ۲.۷ به بالا) استفاده نمایید.

## ۲ محیط Fedora

از آنجا که مخازن رسمی فدورا ممکن است کامپایلر دقیق توصیه‌شده توسط MIT را در خود نداشته باشند، می‌توانید از یکی از دو روش زیر استفاده کنید:  
روش اول (پیشنهادی): استفاده از بسته‌های آماده مخازن فدورا

```
sudo dnf install git make gcc gdb qemu-system-riscv
```

سپس کامپایلرهای متقاطع مربوط به معماری RISC-V را جستجو کنید:

```
sudo dnf search riscv
```

به دنبال بسته‌هایی با عنوان `riscv64-linux-gnu-gcc` و `riscv64-linux-gnu-binutils` بگردید و در صورت وجود، آن‌ها را نصب کنید:

```
sudo dnf install riscv64-linux-gnu-gcc riscv64-linux-gnu-binutils
```

روش دوم: کامپایل دستی زنجیره ابزار رسمی (**RISC-V GNU Toolchain**)  
در صورتی که با بسته‌های پیش‌فرض توزیع با مشکل عدم ناسازگاری مواجه شدید، می‌توانید زنجیره ابزار رسمی را به صورت دستی کامپایل نمایید:

```
git clone https://github.com/riscv-collab/riscv-gnu-toolchain cd  
riscv-gnu-toolchain ./configure --prefix=/opt/riscv make newlib
```

پس از پایان کامپایل، مسیر ابزارها را به متغیر `PATH` خود اضافه نمایید:

```
export PATH=/opt/riscv/bin:$PATH
```

سپس همانند سایر توزیع‌ها، مخزن اصلی را دریافت و اجرا کنید. (بنابر مستندات رسمی xv6 استفاده از کتابخانه `newlib` برای اجرای کدهای سطح هسته توصیه‌شده است).

### ۳ محیط Arch Linux

مخازن توزیع آرچ به طور مستقیم کامپایلرهای رسمی متقاطع مناسب را ارائه می‌دهند. ابتدا سیستم خود را به‌روزرسانی کرده و ابزارهای مورد نیاز را نصب نمایید:

```
sudo pacman -Syu sudo pacman -S git base-devel qemu-emulators-full  
riscv64-elf-gcc riscv64-elf-binutils gdb
```

پس از اتمام مراحل نصب، مخزن پروژه را دریافت و راه‌اندازی کنید:

```
git clone https://github.com/mit-pdos/xv6-riscv.git cd xv6-riscv make qemu
```

### ۴ محیط Windows (از طریق WSL2)

مناسب‌ترین و پایدارترین روش جهت اجرای پروژه روی سیستم‌عامل ویندوز، استفاده از محیط شبیه‌ساز لینوکس WSL2 می‌باشد. ابتدا برنامه PowerShell ویندوز را در حالت دسترسی مدیر (Administrator) باز کرده و دستور زیر را بنویسید:

```
wsl --install
```

سیستم را در صورت نیاز راه‌اندازی مجدد کنید و سپس محیط خط فرمان لینوکس (اوبونتو) را با دستور زیر فراخوانی نمایید:

```
wsl
```

درون ترمینال لینوکس تازه باز شده، مراحل نصب دقیقاً مشابه با بخش اوبونتو / دبیان دنبال می‌شود:

```
sudo apt update sudo apt install git build-essential gdb-multiarch  
qemu-system-misc gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu git clone  
https://github.com/mit-pdos/xv6-riscv.git cd xv6-riscv make qemu
```



❏ تأیید صحت عملکرد نصب

۵

برای اطمینان از صحت نصب زنجیره ابزار و هماهنگی شبیه‌ساز، دستورات زیر را در خط فرمان سیستم خود به صورت مجزا اجرا نمایید:

```
Check cross-compiler (either riscv64-unknown-elf-gcc or riscv64-elf-gcc)
riscv64-unknown-elf-gcc --version Check QEMU qemu-system-riscv64 --version
```

پس از اجرای موفقیت‌آمیز دستور `make qemu` در دایرکتوری اصلی پروژه، خروجی بوت سیستم‌عامل باید مشابه کادر زیر خاتمه یابد که نشان‌دهنده ورود موفق به پوسته (Shell) پیش‌فرض xv6 است:

```
xv6 kernel is booting
init: starting sh
```

### ۳ بخش اول: افزودن System Call جدید

هدف: پیاده‌سازی یک System Call به نام `getpinfo` برای دریافت اطلاعات فرآیندها.

سیستم‌عامل به طور پیش‌فرض اطلاعات محدودی از فرآیندها به کاربر نمایش می‌دهد. شما باید یک فراخوان سیستمی طراحی کنید که اطلاعات مربوط به تمام فرآیندهای موجود در سیستم (مانند PID، وضعیت فرآیند، اولویت، و تعداد بلیت‌ها) را جمع‌آوری کرده و به فضای کاربر (User Space) بازگرداند.

اقدامات لازم:

- تعریف ساختار `struct pinfo` در یک فایل هدر مجزا (مانند `pinfo.h`) که هم در هسته و هم در برنامه کاربردی قابل دسترسی باشد. این ساختار باید فیلدهای لازم برای ذخیره اطلاعات فرآیندها را تعریف کند.

- تعریف شماره `syscall` در فایل‌های مربوطه (مانند `syscall.h`).

- نوشتن بدنه اصلی `sys_getpinfo` در `sysproc.c` یا فایلی مشابه. جهت حفظ سازگاری و امنیت داده‌ها، باید از مکانیزم‌های قفل‌گذاری هسته (مانند قفل جدول فرآیندها) هنگام خواندن اطلاعات استفاده شود تا از بروز شرایط رقابتی (Race Condition) جلوگیری به عمل آید.

- نوشتن یک برنامه کاربردی در سطح سیستم‌عامل (مانند `ps.c`) که این فراخوان سیستمی را صدا زده و نتایج را در خروجی استاندارد چاپ کند.

خروجی مورد انتظار: هنگام اجرای دستور `ps` در خط فرمان `xv6`، لیستی از فرآیندها همراه با اطلاعات کامل آن‌ها نمایش داده شود.

## ۴ بخش دوم: پیاده‌سازی Priority Scheduling

هدف: جایگزینی زمان‌بند Round Robin پیش‌فرض با یک زمان‌بند مبتنی بر اولویت و افزودن فراخوان سیستمی `setpriority`.

به صورت پیش‌فرض، xv6 از الگوریتم Round Robin استفاده می‌کند. شما باید ساختار `proc` (در فایل `proc.h`) را تغییر دهید تا هر فرآیند دارای یک مقدار اولویت عددی بین ۰ تا ۱۰۰ باشد. در این سیستم، عدد کمتر نشان‌دهنده اولویت بالاتر است (عدد ۰ بالاترین اولویت و عدد ۱۰۰ کمترین اولویت در نظر گرفته می‌شود).

اقدامات لازم:

- ویرایش ساختار فرآیند (`struct proc`) برای ذخیره مقدار اولویت.
  - مقداردهی اولیه اولویت پیش‌فرض برای فرآیندهای جدید با مقدار متوسط ۵۰.
  - پیاده‌سازی فراخوان سیستمی `setpriority(int pid, int priority)` برای تغییر اولویت یک فرآیند در حال اجرا بر اساس شناسه‌اش.
  - تغییر تابع `scheduler()` در فایل `proc.c` به طوری که در هر مرحله، پردازنده به فرآیندی که در وضعیت `RUNNABLE` است و بالاترین اولویت (کمترین مقدار عددی) را دارد اختصاص یابد.
  - قانون شکستن گره (Tie-breaking) در صورتی که چند فرآیند آماده به کار اولویت یکسان و بالاتری داشته باشند، زمان‌بند باید زمان پردازنده را به صورت چرخشی (Round Robin) میان آن‌ها تقسیم کند.
  - مسئله گرسنگی (Starvation): برای سادگی پروژه، نیازی به پیاده‌سازی فرآیند پیری پویا (Aging) جهت جلوگیری از گرسنگی نیست و زمان‌بندی بر اساس اولویت مطلق کفایت می‌کند.
- خروجی مورد انتظار: کاربر بتواند با اجرای دستوری نظیر `chpri [pid] [priority]` اولویت فرآیند را تغییر دهد. همچنین زمان‌بند باید رفتار خود را بر اساس این اولویت‌ها تنظیم کرده و فرآیندهای با

اولویت بالاتر پردازنده را زودتر تصاحب کنند. (نیاز به نوشتن برنامه تست برای نشان دادن این رقابت الزامی است).

## ۵ بخش سوم: پیاده‌سازی Lottery Scheduling (Bonus)

هدف: پیاده‌سازی الگوریتم قرعه‌کشی (Lottery) برای تخصیص زمان پردازنده به صورت احتمالی بر اساس تعداد بلیت‌ها (Tickets).

در زمان‌بندی Lottery، به هر فرآیند تعدادی بلیت اختصاص داده می‌شود. زمان‌بند در هر بار انتخاب فرآیند، یک عدد تصادفی بین ۱ تا مجموع کل بلیت‌های فرآیندهای `RUNNABLE` تولید کرده و فرآیند برنده را انتخاب می‌کند. اقدامات لازم:

- افزودن متغیر `tickets` به `struct proc`. فرآیندها به طور پیش‌فرض هنگام ایجاد باید دارای ۱ بلیت باشند. در هنگام فراخوانی `fork()`، فرآیند فرزند باید تعداد بلیت‌های فرآیند والد را به ارث ببرد.

- پیاده‌سازی یک مولد اعداد شبه‌تصادفی (PRNG) ساده (مانند الگوریتم LCG) در هسته xv6 که از متغیر تیک‌های سیستم (`ticks`) به عنوان هسته اولیه (Seed) استفاده کند.

- پیاده‌سازی فراخوان سیستمی `settickets(int number)` برای تعیین تعداد بلیت‌های یک فرآیند.

- تغییر مجدد تابع `scheduler()` برای پیاده‌سازی منطق قرعه‌کشی به جای استفاده از اولویت مطلق. در این حالت، فرآیندی که بلیت بیشتری دارد، شانس بیشتری برای تصاحب پردازنده (CPU) خواهد داشت.

خروجی مورد انتظار: فرآیندهایی که بلیت بیشتری (مثلاً ۱۰۰ بلیت در برابر ۱۰ بلیت) دریافت کرده‌اند، در طول زمان اجرای سیستم، دفعات بیشتری توسط پردازنده انتخاب شده و زودتر به اتمام می‌رسند. این رفتار باید در مستندات شما و توسط برنامه‌های تستی که می‌نویسید قابل مشاهده و اثبات باشد.

راهنمای مهم: همزیستی و سوئیچ بین زمان‌بندها  
کدهای مربوط به زمان‌بندها باید به گونه‌ای پیاده‌سازی شوند که ساختار زمان‌بند اصلی بدون نیاز به بازنویسی مجدد کد، از طریق فایل `Makefile` و در زمان کامپایل قابل تغییر باشد. برای این

کار باید از دستورات عمل‌های پیش‌پردازنده سی (مانند `#ifdef`) استفاده نمایید.  
به عنوان نمونه، ساختار کامپایل باید با تعریف متغیر `SCHEDULER` به صورت زیر پشتیبانی شود:

```
# To run with Priority Scheduler: make qemu SCHEDULER=PRIORITY  
# To run with Lottery Scheduler: make qemu SCHEDULER=LOTTERY
```

## ۶ معیارهای پذیرش و بارمبندی (Acceptance Criteria)

ارزیابی پروژه به صورت دقیق بر اساس معیارهای فنی زیر و در جلسه دفاع حضوری/آنلاین انجام خواهد شد. مجموع نمرات این پروژه شامل ۲۰ نمره پایه به همراه ۴ نمره امتیازی (بونوس) می‌باشد.

### ۱-۶ بخش اول: فراخوان سیستمی `getpininfo` و برنامه کاربردی `ps` (۶ نمره)

- صحت تعریف و ثبت سیستم‌کال (۲ نمره): تعریف صحیح شماره، پروتوتایپ و متدهای لازم در فایل‌های هسته و فضای کاربر بدون خطا در زمان کامپایل.
- طراحی ایمن و مستقل از نوع داده (۲ نمره): تعریف ساختار `struct pininfo` در یک فایل هدر مجزا و انتقال ایمن داده‌ها از فضای هسته به فضای کاربر با استفاده از قفل‌های سیستم (`ptable lock`) جهت جلوگیری از بروز شرایط رقابتی.
- برنامه کاربردی `ps` (۲ نمره): چاپ دقیق و خوانای اطلاعات فرآیندها به صورت ستونی در خروجی استاندارد.

### ۲-۶ بخش دوم: زمان‌بندی مبتنی بر اولویت و برنامه کاربردی `chpri` (۱۰ نمره)

- تغییر ساختار فرآیند و مقداردهی اولیه (۲ نمره): افزودن متغیر اولویت به ساختار `proc` و مقداردهی اولیه درست در زمان تخصیص فرآیند جدید (`allocproc`).
- سیستم‌کال `setpriority` و ابزار `chpri` (۳ نمره): پیاده‌سازی درست تغییر اولویت فرآیند بر اساس PID در فضای هسته و برنامه تغییر اولویت از سمت خط فرمان کاربر.
- مکانیزم زمان‌بند اولویت و رفتار زمان اجرا (۴ نمره): انتخاب درست فرآیند با کمترین مقدار عددی اولویت و توزیع عادلانه پردازنده به روش Round Robin بین فرآیندهای هم‌اولویت.
- پایداری سیستم (۱ نمره): عدم وجود هرگونه بن‌بست (Deadlock)، کرش، یا مصرف نادرست منابع در حین تغییر اولویت فرآیندها تحت بار سنگین پردازشی.

### ۳-۶ بخش سوم (امتیازی): زمان بندی قرعه کشی (Lottery) (۴+ نمره بونوس)

- مکانیزم وراثت بلیت ها و سیستم کال (۵.۱ نمره): ارث بری صحیح بلیت ها از والد به فرزند در زمان فراخوانی `fork()` و صحت کارکرد فراخوان سیستمی `settickets`.
- پیاده سازی مولد اعداد تصادفی (۱ نمره): عدم استفاده از کتابخانه های خارجی و پیاده سازی درست یک PRNG ساده درون هسته با تکیه بر مقادیر زمان سیستم (`ticks`).
- الگوریتم زمان بند قرعه کشی و اثبات آماری (۵.۱ نمره): کارکرد درست تخصیص تصادفی پردازنده بر اساس سهم بلیت هر فرآیند و ارائه گزارش/مستندات آماری که نشان دهد توزیع زمان پردازنده با نسبت بلیت ها همخوانی دارد.

### ۴-۶ معیارهای عمومی، گیت و جلسه ارائه (۴ نمره)

- یکپارچه سازی و استفاده از پرچم های کامپایل (۲ نمره): پیاده سازی پروژه به صورت ساختاریافته به گونه ای که با تغییر متغیرهای کامپایل در `Makefile` بتوان نسخه پیش فرض، اولویت دار و یا قرعه کشی را مستقلاً اجرا نمود.
- تاریخچه Git و کدهای تمیز (۲ نمره): رعایت اصول کدنویسی تمیز، استفاده از کامنت های توضیحی برای بخش های تغییر یافته هسته و وجود یک تاریخچه کامیت منظم و معتبر در گیت هاب که نشان دهنده فرآیند گام به گام پیاده سازی باشد.
- بسیار مهم - نحوه ارزیابی در جلسه ارائه: نمره نهایی پروژه منوط به پاسخگویی فردی اعضای گروه به سوالات فنی زمان دفاع است. ارائه و اجرای موفق برنامه های تست اختصاصی که برای اثبات هر بخش نوشته اید در جلسه دفاع الزامی است. در صورت عدم تسلط کامل هر یک از اعضا بر کدهای ارائه شده یا عدم توانایی در تحلیل کدهای نوشته شده، نمره آن بخش برای فرد مربوطه منظور نخواهد شد.



## ۷ منابع پیشنهادی

برای درک بهتر معماری سیستم‌عامل xv6 و کدهای هسته، مطالعه منابع زیر به شدت توصیه می‌شود. این منابع می‌توانند به شما در رفع باگ‌ها و درک منطق سیستم‌کال‌ها کمک شایانی کنند:

- کتاب رسمی **xv6** (نسخه **RISC-V**): این کتاب مرجع اصلی سیستم‌عامل xv6 است. برای این پروژه، خواندن فصل‌های مربوط به Traps و Scheduling بسیار راهگشا خواهد بود:

<https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf>

- ویدیوهای آموزشی دانشگاه **MIT** (درس **6.S081**): تدریس رسمی این سیستم‌عامل توسط توسعه‌دهندگان آن به صورت رایگان در یوتیوب موجود است. جلساتی که به تشریح عملکرد تابع `scheduler` می‌پردازند را حتماً مشاهده کنید.
- مستندات معماری **RISC-V**: برای درک بهتر نحوه تعامل هسته با سخت‌افزار و مفهوم رجیسترها (در صورت علاقه به یادگیری عمیق‌تر).